

Name: Vũ Đại Dương

ID: 23520359

Class: IT007.P28.2

OPERATING SYSTEM LAB 6'S REPORT

SUMMARY

Task		Status	Page
Section 6.4	Task 1	Done	11
	Task 2	Done	13
	Task 3	Done	15
	Task 4	Done	18
	Task 5	Done	19
...	...		
	...		
	...		

Self-scores: 10/10

Note: Export file to **PDF and name the file by following format:
Student ID_LABx.pdf*

Section 6.4

- ✚ Vì mỗi yêu cầu trong bài LAB này có thể ghép thành 1 chương trình hoàn chỉnh nên ta có thể sử dụng 1 source code cho cả bài để thực hiện tất cả yêu cầu

✚ SOURCE CODE:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <unistd.h>
5  #include <sys/types.h>
6  #include <sys/wait.h>
7  #include <fcntl.h>
8  #include <signal.h>
9  #include <termios.h>
10
11
12  #define MAX_LINE 80
13
14  int number_of_args;
15  int fdi, fdo;
16  char *args[MAX_LINE / 2 + 1];
17  int SaveStdin, SaveStdout;
18
19  void enableRawMode(struct termios *orig_termios)
20  {
21      struct termios raw;
22      tcgetattr(STDIN_FILENO, orig_termios);
23      raw = *orig_termios;
24      raw.c_lflag &= ~(ICANON | ECHO); // Tắt canonical mode và echo
25      tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw);
26  }
27
28  void disableRawMode(struct termios *orig_termios)
29  {
30      tcsetattr(STDIN_FILENO, TCSAFLUSH, orig_termios);
31  }
32
33  void Allocate_Memory(char *args_array[])
34  {
35      for (int i = 0; i < MAX_LINE / 2 + 1; i++)
36      {
37          args_array[i] = (char *)malloc(sizeof(char) * MAX_LINE / 2);
38      }
39  }
40
41  void EnterCommand(char command[], char history_command[][MAX_LINE], int count_HF, int *index)
42  {
43      struct termios orig_termios;
44      enableRawMode(&orig_termios);
45  }
```

Hình 1: Source code từ dòng 1 - 45

```

46     int pos = 0;
47     char ch;
48
49     printf("it007sh>");
50     fflush(stdout);
51
52     while (1)
53     {
54         ch = getchar();
55
56         if (ch == 27) // ESC
57         {
58             char next1 = getchar();
59             char next2 = getchar();
60             if (next1 == '[')
61             {
62                 if (next2 == 'A') // UP
63                 {
64                     if (*index > 0)
65                         (*index)--;
66
67                     printf("\33[2K\r");
68                     printf("it007sh>%s", history_command[*index]);
69                     fflush(stdout);
70
71                     strcpy(command, history_command[*index]);
72                     pos = strlen(command);
73                 }
74                 else if (next2 == 'B') // DOWN
75                 {
76                     if (*index < count_HF - 1)
77                         (*index)++;
78
79                     printf("\33[2K\r");
80                     printf("it007sh>%s", history_command[*index]);
81                     fflush(stdout);
82
83                     strcpy(command, history_command[*index]);
84                     pos = strlen(command);
85                 }
86             }
87         }
88         else if (ch == '\n')
89         {
90             command[pos] = '\0';

```

Hình 2: Source code từ dòng 46 - 90

```

91         printf("\n");
92         break;
93     }
94     else if (ch == 127 || ch == 8) // backspace
95     {
96         if (pos > 0)
97         {
98             pos--;
99             printf("\b \b");
100             fflush(stdout);
101         }
102     }
103     else
104     {
105         if (pos < MAX_LINE - 1)
106         {
107             command[pos++] = ch;
108             putchar(ch);
109             fflush(stdout);
110             *index = count_HF;
111         }
112     }
113 }
114
115 disableRawMode(&orig_termios);
116 }
117
118 void Tokernizer(char *tokens[], char *source, char *delim, int *num_of_words)
119 {
120     char *p = strtok(source, delim);
121     int count = 0;
122     while (p != NULL)
123     {
124         strcpy(tokens[count], p);
125         count++;
126         p = strtok(NULL, delim);
127     }
128     *num_of_words = count;
129 }
130
131 void control_sig(int sign)
132 {
133     printf("\n");
134     fflush(stdout);
135 }

```

Hình 3: Source code từ dòng 91 - 135

```

136
137 void redirect_output()
138 {
139     SaveStdout = dup(STDOUT_FILENO);
140     fdo = open(args[number_of_args - 1], O_CREAT | O_WRONLY | O_TRUNC, 0644);
141     dup2(fdo, STDOUT_FILENO);
142     free(args[number_of_args - 2]);
143     args[number_of_args - 2] = NULL;
144 }
145
146 void redirect_input()
147 {
148     SaveStdin = dup(STDIN_FILENO);
149     fdi = open(args[number_of_args - 1], O_RDONLY);
150     dup2(fdi, STDIN_FILENO);
151     free(args[number_of_args - 2]);
152     args[number_of_args - 2] = NULL;
153 }
154
155 void RestoreOut()
156 {
157     close(fdo);
158     dup2(SaveStdout, STDOUT_FILENO);
159     close(SaveStdout);
160 }
161
162 void RestoreIn()
163 {
164     close(fdi);
165     dup2(SaveStdin, STDIN_FILENO);
166     close(SaveStdin);
167 }
168
169 void Execute_pile(char *parsed[], char *parsedpipe[])
170 {
171     pid_t PID1, PID2;
172     int fd[2];
173
174     PID1 = fork();
175     if (PID1 < 0)
176     {
177         printf("\nLỗi không thể tạo tiến trình con");
178         return;
179     }
180

```

Hình 4: Source code từ dòng 136 - 180

```

181     if (PID1 == 0)
182     {
183         if (pipe(fd) < 0)
184         {
185             printf("\nTạo ống thất bại");
186             return;
187         }
188
189         PID2 = fork();
190         if (PID2 < 0)
191         {
192             printf("\nLỗi không thể tạo tiến trình con");
193             exit(0);
194         }
195
196         if (PID2 == 0)
197         {
198             close(fd[0]);
199             dup2(fd[1], STDOUT_FILENO);
200             if (execvp(parsed[0], parsed) < 0)
201             {
202                 printf("\nKhông thể thực thi");
203                 exit(0);
204             }
205         }
206         else
207         {
208             wait(NULL);
209             close(fd[1]);
210             dup2(fd[0], STDIN_FILENO);
211             if (execvp(parsedpipe[0], parsedpipe) < 0)
212             {
213                 printf("\nKhông thể thực thi");
214                 exit(0);
215             }
216         }
217     }
218     else
219     {
220         wait(NULL);
221     }
222 }
223
224 int Find_pile_char(char *cmd, char *cmdpiped[])
225 {
226     for (int i = 0; i < 2; i++)
227     {
228         cmdpiped[i] = strsep(&cmd, "|");
229         if (cmdpiped[i] == NULL)
230             break;
231     }
232     return (cmdpiped[1] != NULL);
233 }
234

```

Hình 5: Source code từ dòng 181 - 234

```

235 void parseCommandLine(char *cmd, char *parsedArg[])
236 {
237     int i = 0;
238     char *token;
239     while ((token = strsep(&cmd, " ")) != NULL)
240     {
241         if (strlen(token) > 0)
242         {
243             parsedArg[i++] = token;
244         }
245     }
246     parsedArg[i] = NULL;
247 }
248
249 int ExecuteString(char *cmd, char *parsed[], char *parsedpipe[])
250 {
251     char *cmdpiped[2];
252     int piped = Find_pile_char(cmd, cmdpiped);
253     if (piped)
254     {
255         parseCommandLine(cmdpiped[0], parsed);
256         parseCommandLine(cmdpiped[1], parsedpipe);
257     }
258     return piped;
259 }
260
261 int main(void)
262 {
263     int count_HF = 0;
264     int should_run = 1;
265     char command[MAX_LINE];
266     char history_command[MAX_LINE][MAX_LINE];
267     char *First_Command[MAX_LINE / 2 + 1];
268     char *Second_command[MAX_LINE / 2 + 1];
269     int iPileExe = 0;
270     pid_t pid;
271     int iRedirectOut = 0, iRedirectIn = 0;
272
273     signal(SIGINT, control_sig);
274
275     while (should_run)
276     {
277         int history_index = count_HF;
278         do {
279             EnterCommand(command, history_command, count_HF, &history_index);
280             while (command[0] == 0);

```

Hình 6: Source code từ dòng 235 - 280

```

282  if (strcmp(command, "HF") == 0)
283  {
284      if (count_HF == 0)
285      {
286          printf("NULL\n");
287          continue;
288      }
289      else
290      {
291          for (int i = 0; i < count_HF; i++)
292          {
293              printf("%s\n", history_command[i]);
294          }
295          continue;
296      }
297  }
298  else
299  {
300      strcpy(history_command[count_HF++], command);
301  }
302
303  iFileExe = ExecuteString(command, First_Command, Second_command);
304
305  if (iFileExe == 0)
306  {
307      Allocate_Memory(args);
308      strcpy(command, history_command[count_HF - 1]);
309      Tokernizer(args, command, " ", &number_of_args);
310      free(args[number_of_args]);
311      args[number_of_args] = NULL;
312
313      if (strcmp(args[0], "exit") == 0)
314          break;
315
316      else if (strcmp(args[0], "~") == 0)
317      {
318          char *homedir = getenv("HOME");
319          printf("Home : %s\n", homedir);
320      }
321      else if (strcmp(args[0], "cd") == 0)
322      {
323          char dir[MAX_LINE];
324          strcpy(dir, args[1]);
325          if (strcmp(dir, "~") == 0)
326          {
327              strcpy(dir, getenv("HOME"));
328          }

```

Hình 7: Source code từ dòng 282 - 328


```

329     chdir(dir);
330     printf("Thư mục hiện tại : ");
331     getcwd(dir, sizeof(dir));
332     printf("%s\n", dir);
333 }
334 else
335 {
336     int isBackground = strcmp(args[number_of_args - 1], "&") == 0 ? 1 : 0;
337     if (isBackground)
338     {
339         free(args[number_of_args - 1]);
340         args[number_of_args - 1] = NULL;
341     }
342
343     if ((number_of_args > 1) && strcmp(args[number_of_args - 2], ">") == 0)
344     {
345         redirect_output();
346         iRedirectOut = 1;
347     }
348     else if ((number_of_args > 1) && strcmp(args[number_of_args - 2], "<") == 0)
349     {
350         redirect_input();
351         iRedirectIn = 1;
352     }
353
354     pid = fork();
355     if (pid < 0)
356     {
357         fprintf(stderr, "Lỗi không thể tạo tiến trình con\n");
358         return -1;
359     }
360
361     if (pid == 0)
362     {
363         int ret = execvp(args[0], args);
364         if (ret == -1)
365         {
366             printf("Lệnh không hợp lệ\n");
367         }
368         exit(ret);
369     }
370     else
371     {
372         if (!isBackground)
373         {
374             while (wait(NULL) != pid);

```

Hình 8: Source code từ dòng 329 - 374

```

375         if (iRedirectOut)
376         {
377             RestoreOut();
378             iRedirectOut = 0;
379         }
380         if (iRedirectIn)
381         {
382             RestoreIn();
383             iRedirectIn = 0;
384         }
385     }
386 }
387 }
388 }
389 else
390 {
391     Execute_pile(First_Command, Second_command); }
392 }
393 return 0;
394 }

```

Hình 9: Source code từ dòng 375 - 395

- ❖ Tại vì các hình ảnh source code quá dài nên để có thể thuận tiện cho thầy theo dõi chương trình nên em sẽ gắn link source code tham khảo tại đây:

https://github.com/VuDaiDuong-23520359/OS_ubuntu/blob/DesktopUIT/TH/lab6/main.c

Giải thích các hàm trong code:

- **Hàm Allocate_Memory:** Dùng để cấp phát bộ nhớ cho mảng, trong chương trình này là cấp phát bộ nhớ có kích thước MAX_LINE/2 và con trỏ tới mảng này qua câu lệnh `args_array[i] = (char *)malloc(sizeof(char) * MAX_LINE /2)`
- **Hàm EnterCommand:** Nhận lệnh từ bàn phím qua stdin và loại bỏ ký tự newline `\n` ở cuối chuỗi.
- **Hàm Tokernizer:** Tách chuỗi source thành các từ con (tokens) bằng các ký tự phân cách delim (thường là dấu cách) và lưu vào tokens[]. Biến chuỗi lệnh nhập từ người dùng thành mảng các đối số có thể dùng với `execvp()`.
- **Hàm control_sig:** Xử lý tín hiệu SIGINT (từ Ctrl+C), chỉ in dòng mới và không kết thúc shell. Dùng để tránh thoát khỏi shell khi người dùng nhấn Ctrl+C.
- **Hàm redirect_output:** Chuyển hướng đầu ra chuẩn (stdout) của tiến trình hiện tại sang ghi vào file (tên file là đối số cuối cùng). Dùng để thực hiện lệnh có chuyển hướng đầu ra, ví dụ `ls > out.txt`. Sử dụng `dup2()` để chuyển hướng đầu ra.
- **Hàm redirect_input:** Chuyển hướng đầu vào chuẩn (stdin) của tiến trình từ một file thay vì từ bàn phím. Dùng để thực hiện lệnh đọc từ file, ví dụ `sort < input.txt`. Sử dụng `dup2()` để chuyển hướng đầu vào.

- **Hàm RestoreOut:** Khôi phục lại stdout ban đầu sau khi đã chuyển hướng ra file. Tiếp tục hiển thị lên màn hình các lệnh sau khi chuyển hướng xong.
- **Hàm RestoreIn:** Khôi phục lại stdin ban đầu sau khi đã chuyển hướng từ file. Trở về đọc từ bàn phím sau khi thực hiện lệnh dùng input từ file.
- **Hàm Execute_pile:** Thực thi hai lệnh được nối với nhau bằng pipe |, chia thành hai tiến trình: Tiến trình 1 ghi ra pipe, Tiến trình 2 đọc từ pipe và thực thi.
Ví dụ: `ls | grep txt` – lệnh `ls` xuất ra sẽ được truyền làm đầu vào cho `grep`
- **Hàm Find_pile_char:** Kiểm tra xem chuỗi lệnh `cmd` có chứa ký tự pipe (|) hay không, nếu có thì tách thành hai phần và lưu vào `cmdpiped[]`. Trả về 1 nếu có pipe, 0 nếu không có để xác định có cần xử lý pipe hay không.
- **Hàm parseCommandLine:** Tách chuỗi `cmd` thành các phần tử nhỏ (các đối số) bằng khoảng trắng và lưu vào `parsedArg[]`. Dùng để chuẩn bị đối số để truyền vào `execvp()`.
- **Hàm ExecuteString:** Gọi `Find_pile_char()` để kiểm tra có pipe không, sau đó tách hai phần lệnh nếu có. Trả về 1 nếu có pipe, 0 nếu là lệnh thường. Dùng để giúp hàm `main()` biết nên dùng `execvp()` trực tiếp hay gọi `Execute_pile()`.

1. Thực thi lệnh trong tiến trình con

```

vudaiduong-23520359@VuDuong-32:~/OS_ubuntu/TH/lab6$ ./m
it007sh>echo abc
abc
it007sh>cd ..
Thư mục hiện tại : /home/vudaiduong-23520359/OS_ubuntu/TH
it007sh>cd lab6
Thư mục hiện tại : /home/vudaiduong-23520359/OS_ubuntu/TH/lab6
it007sh>touch vduong.txt
it007sh>echo abc > vduong.txt
it007sh>cat vduong.txt
abc
it007sh>abc
Lệnh không hợp lệ
it007sh>

```

Hình 10: Nhập các câu lệnh và dấu nhắc `it007sh>` sau khi thực hiện xong các câu lệnh

```

● vudaiduong-23520359@VuDuong-32:~/OS_ubuntu/TH/lab6$ ./m
it007sh>echo abc
abc
it007sh>cd ..
Thư mục hiện tại : /home/vudaiduong-23520359/OS_ubuntu/TH
it007sh>cd lab6
Thư mục hiện tại : /home/vudaiduong-23520359/OS_ubuntu/TH/lab6
it007sh>touch vduong.txt
it007sh>echo abc > vduong.txt
it007sh>cat vduong.txt
abc
it007sh>abc
Lệnh không hợp lệ
it007sh>ls
a m main.c vduong.txt
it007sh>exit
○ vudaiduong-23520359@VuDuong-32:~/OS_ubuntu/TH/lab6$

```

Hình 12: Gõ exit để thoát khỏi chương trình

✚ Giải thích kết quả:

- Khi thực hiện lệnh **cat vduong.txt**, chương trình đầu tiên sẽ sao chép lệnh này vào mảng **history_command** để lưu lại lịch sử. Sau đó, nó kiểm tra xem lệnh có chứa ký tự pipe (|) hay không bằng hàm **ExecuteString()**. Do đây là một lệnh đơn giản, không có pipe nên giá trị **iPipeExe** được gán bằng 0. Tiếp theo, chương trình cấp phát bộ nhớ cho mảng args và sử dụng hàm **Tokernizer()** để tách lệnh thành các đối số riêng biệt. Lúc này, **args[0] = "cat"** và **args[1] = "vduong.txt"**.

- Sau khi loại trừ các lệnh đặc biệt như exit, cd, hoặc ~, chương trình tiến hành tạo một tiến trình con bằng lệnh **fork()**. Trong tiến trình con, hàm **execvp()** được gọi để thực thi lệnh **cat vduong.txt**. Nếu thực thi thành công, nội dung trong file sẽ được in ra màn hình. Trong trường hợp file không tồn tại hoặc lệnh không hợp lệ, chương trình sẽ thông báo lỗi. Sau khi tiến trình con kết thúc, tiến trình cha sẽ đợi bằng **wait()**, khôi phục lại đầu vào/ra nếu có chuyển hướng, và in lại dấu nhắc it007sh> để chờ người dùng nhập lệnh mới. Trong ví dụ này, kết quả hiển thị trên màn hình là dòng chữ **"abc"**, chính là nội dung của file **vduong.txt**. Và muốn thoát khỏi chương trình thì ta nhập **exit** sau dấu nhắc.

2. Tạo tính năng sử dụng lại câu lệnh gần đây

```
○ vudaiduong-23520359@VuDuong-32:~/OS_ubuntu/TH/lab6$ ./m
it007sh>echo abc
abc
it007sh>ls -l
total 40
-rw-rw-r-- 1 vudaiduong-23520359 vudaiduong-23520359  0 May 24 10:29 a
-rwxrwxr-x 1 vudaiduong-23520359 vudaiduong-23520359 22056 May 24 11:19 m
-rw-rw-r-- 1 vudaiduong-23520359 vudaiduong-23520359 9565 May 24 11:29 main.c
-rw-rw-r-- 1 vudaiduong-23520359 vudaiduong-23520359  4 May 24 10:56 vduong.txt
it007sh>pwd
/home/vudaiduong-23520359/OS_ubuntu/TH/lab6
it007sh>HF
echo abc
ls -l
pwd
it007sh>
```

Hình 13: Nhập các câu lệnh và nhập HF để hiện lịch sử câu lệnh

```
it007sh>HF
echo abc
ls -l
pwd
it007sh>
```

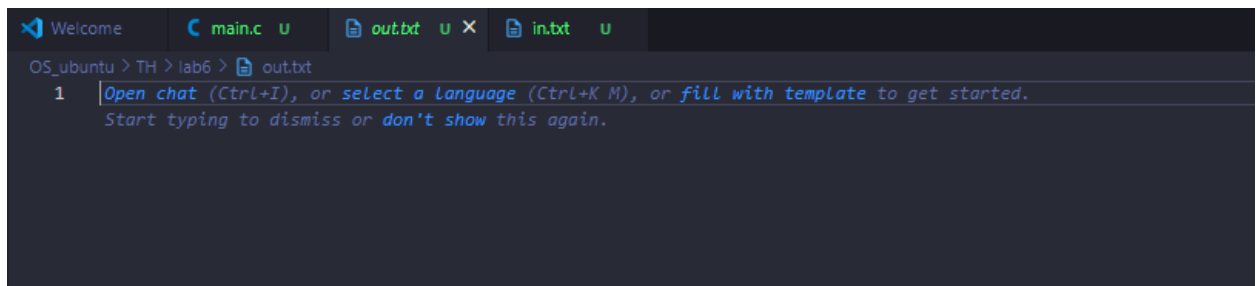
Hình 14: Thao tác để recall các câu lệnh

🔧 Giải thích kết quả:

- Ban đầu, ta thực hiện câu lệnh `echo abc`, chương trình sẽ lưu lệnh này trong mảng **history_command** và biến đếm số lệnh lúc này **Count_HF** sẽ là 1
- Sau đó ta thực hiện lệnh thứ 2 `ls -l`, chương trình sẽ lưu lệnh này trong mảng **history_command** và biến đếm số lệnh lúc này **Count_HF** sẽ là 2
- Tiếp theo ta thực hiện lệnh thứ 3 là `pwd`, chương trình sẽ lưu lệnh này trong mảng **history_command** và biến đếm số lệnh lúc này **Count_HF** sẽ là 3.
- Cuối cùng ta nhập vào bàn phím HF, chương trình kiểm tra và thấy chuỗi trên đang yêu cầu lịch sử câu lệnh. Kết quả chương trình in ra các câu lệnh đã sử dụng là **echo abc, ls -l, pwd** (hình 13)
- Và để thực hiện các thao tác lên xuống thì em đã xây dựng chương trình gán cứ mỗi thao tác lên sẽ hiện ký tự 'A' cho thao tác lên (↑) và ký tự 'B' cho thao tác xuống (↓). Sử dụng rawmode để bắt đúng các phím. Khi sử dụng phím lên (xuống) để recall lệnh, prompt sẽ xóa lệnh hiện tại, thay lệnh trước (sau) đó vào. Nhấn Enter để chạy lệnh

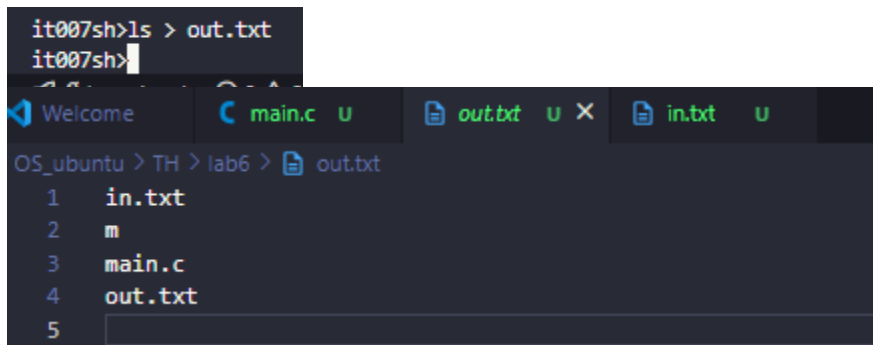
3. Chuyển hướng vào ra

a. Chuyển hướng đầu ra



```
Welcome  main.c  out.txt  in.txt
OS_ubuntu > TH > lab6 > out.txt
1  Open chat (Ctrl+I), or select a language (Ctrl+K M), or fill with template to get started.
   Start typing to dismiss or don't show this again.
```

Hình 17: Nội dung file `out.txt` trước khi thực hiện câu lệnh



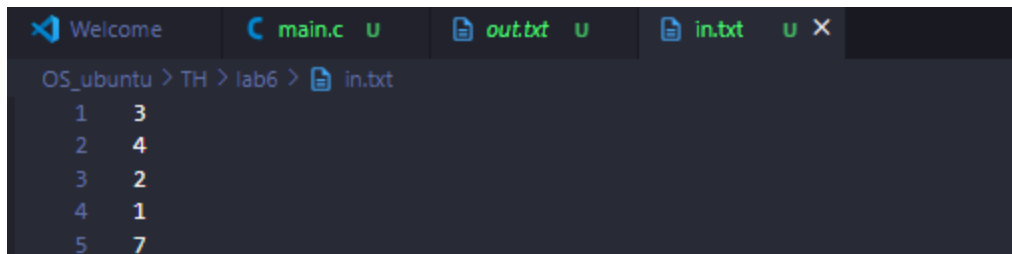
```
it007sh>ls > out.txt
it007sh>
Welcome  main.c  out.txt  in.txt
OS_ubuntu > TH > lab6 > out.txt
1  in.txt
2  m
3  main.c
4  out.txt
5
```

Hình 18: Nội dung file `out.txt` sau khi thực hiện câu lệnh

🔧 Giải thích kết quả:

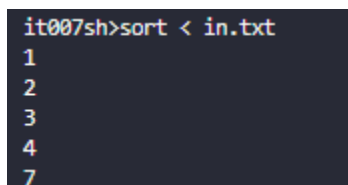
- Khi người dùng nhập lệnh `ls > out.txt`, chương trình sẽ sao chép lệnh vào mảng **history_command** và kiểm tra xem lệnh có chứa pipe (`|`) hay không. Vì đây là lệnh đơn giản, không có pipe nên **iPipeExe** được gán bằng 0. Sau đó, chương trình cấp phát bộ nhớ và tách lệnh thành các đối số qua hàm **Tokernizer()**, tạo ra các phần tử như `args[0] = "ls"`, `args[1] = ">"`, `args[2] = "out.txt"`.
- Chương trình tiếp tục kiểm tra và phát hiện có chuyển hướng đầu ra (`>`), nên sẽ gọi hàm **redirect_output()**. Trong hàm này, một bản sao của `stdout` được lưu lại, sau đó chương trình mở file `out.txt` ở chế độ ghi, tạo mới nếu chưa tồn tại. File descriptor của **out.txt** sẽ được sao chép vào **STDOUT_FILENO** bằng **dup2()**, nghĩa là mọi nội dung được in ra sau đó sẽ ghi vào file thay vì màn hình.
- Cuối cùng, tiến trình con được tạo ra bằng **fork()** và trong tiến trình con, **execvp()** thực thi lệnh `ls`. Kết quả là nội dung thư mục hiện tại được ghi vào file **out.txt**. Sau khi tiến trình kết thúc, chương trình gọi **RestoreOut()** để phục hồi lại luồng xuất chuẩn và in lại dấu nhắc `it007sh>` để chờ lệnh tiếp theo.

b. Chuyển hướng đầu vào



```
Welcome  main.c  out.txt  in.txt
OS_ubuntu > TH > lab6 > in.txt
1 3
2 4
3 2
4 1
5 7
```

Hình 19: Nội dung file `in.txt` trước khi thực hiện câu lệnh



```
it007sh>sort < in.txt
1
2
3
4
7
```

Hình 20: Kết quả sau khi chạy câu lệnh với ngõ vào là file `in.txt`

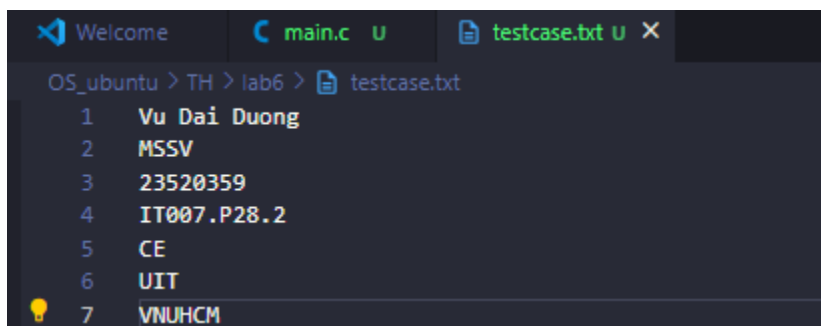
Giải thích kết quả:

Với lệnh **sort < in.txt**, chương trình cũng tiến hành sao chép lệnh vào mảng **history_command**, kiểm tra pipe và gán **iPipeExe = 0**. Sau khi cấp phát bộ nhớ và tách lệnh thành các đối số **args[0] = "sort"**, **args[1] = "<"**, **args[2] = "in.txt"**, chương trình phát hiện có chuyển hướng đầu vào (<), nên gọi hàm **redirect_input()**.

Trong hàm này, một bản sao của stdin được lưu lại, sau đó file **in.txt** được mở ở chế độ chỉ đọc. File descriptor này sẽ được sao chép vào **STDIN_FILENO** bằng **dup2()**, giúp lệnh sort nhận dữ liệu đầu vào từ file thay vì bàn phím. Tiến trình con được tạo ra và thực thi lệnh sort bằng **execvp()**. Kết quả là nội dung trong file **in.txt** được sắp xếp và in ra màn hình theo thứ tự tăng dần. Sau khi kết thúc, stdin được khôi phục lại qua **RestoreIn()** và shell hiện lại **it007sh>**.

4. Giao tiếp sử dụng cơ chế đường ống

- Testcase 1:



```
Welcome  main.c  testcase.txt
OS_ubuntu > TH > lab6 > testcase.txt
1 Vu Dai Duong
2 MSSV
3 23520359
4 IT007.P28.2
5 CE
6 UIT
7 VNUHCM
```

```

it007sh>cat testcase.txt
Vu Dai Duong
MSSV
23520359
IT007.P28.2
CE
UIT
it007sh>cat testcase.txt | sort
23520359
CE
IT007.P28.2
MSSV
UIT
VNUHCM
Vu Dai Duong
it007sh>

```

Hình 21: Kết quả chạy testcase 1 với lệnh `cat testcase.txt`

✚ Giải thích kết quả:

- Khi thực hiện lệnh **`cat test.txt | sort`**, chương trình sẽ sao chép lệnh vào mảng **`history_command`** và kiểm tra sự xuất hiện của ký tự `|` bằng hàm **`ExecuteString()`**. Do đây là lệnh có sử dụng pipe nên giá trị **`iPipeExe`** được gán là 1. Lệnh sau đó được tách thành hai phần: phần trước pipe (**`cat test.txt`**) và phần sau pipe (**`sort`**) thông qua mảng **`First_Command[]`** và **`Second_command[]`**.
- Chương trình sau đó gọi hàm **`Execute_pile()`** để thực hiện cơ chế giao tiếp giữa hai tiến trình. Trong hàm này, chương trình đầu tiên tạo tiến trình con p1 bằng **`fork()`**. Bên trong p1, một ống pipe (**`pipe(fd)`**) được thiết lập để kết nối dữ liệu giữa hai tiến trình con. Tiếp tục, p1 tạo thêm một tiến trình con p2.
- Tiến trình p2 thực hiện lệnh đầu tiên (**`cat test.txt`**). Nó đóng đầu đọc của pipe, sao chép đầu ghi pipe vào **`STDOUT_FILENO`** bằng **`dup2()`**, và gọi **`execvp()`** để thực thi **`cat`**. Kết quả của lệnh này được ghi vào đầu ghi của pipe thay vì in ra màn hình.
- Khi p2 kết thúc, p1 tiếp tục thực hiện lệnh **`sort`**. Trước đó, p1 đóng đầu ghi của pipe, sao chép đầu đọc vào **`STDIN_FILENO`** để nhận dữ liệu từ p2, sau đó thực thi **`sort`** bằng **`execvp()`**. Nhờ đó, đầu ra của **`cat test.txt`** sẽ được truyền trực tiếp sang **`sort`** và kết quả cuối cùng được in ra màn hình theo thứ tự sắp xếp.

- Testcase 2:

```

it007sh>ls | grep test
testcase.txt
it007sh>

```

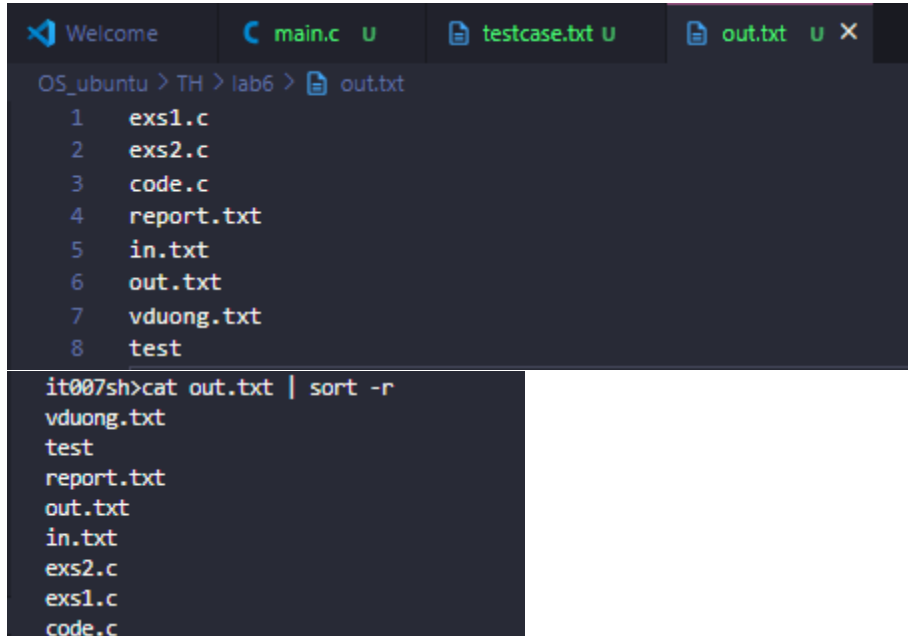
Hình 22: Kết quả chạy testcase 2 với lệnh `ls | grep test`

✚ Giải thích kết quả:

- Với lệnh **`ls | grep test`**, chương trình cũng phát hiện có pipe, tách lệnh thành **`ls`** và **`grep test`**. Quá trình tạo tiến trình và ống pipe cũng diễn ra như ví dụ trước. Tiến trình p2 thực hiện **`ls`**, ghi kết quả liệt kê thư mục vào pipe, còn tiến trình p1 thực hiện **`grep test`**, đọc dữ liệu từ pipe và lọc ra những dòng có chứa từ "test".

- Kết quả cuối cùng là chương trình chỉ in ra những file hoặc thư mục có tên chứa từ "test".
Lệnh này rất thực tế và được sử dụng phổ biến để lọc kết quả đầu ra.

- Testcase 3:



```

Welcome  main.c U  testcase.txt U  out.txt U X
OS_ubuntu > TH > lab6 > out.txt
1  exs1.c
2  exs2.c
3  code.c
4  report.txt
5  in.txt
6  out.txt
7  vduong.txt
8  test

it007sh>cat out.txt | sort -r
vduong.txt
test
report.txt
out.txt
in.txt
exs2.c
exs1.c
code.c

```

Hình 23: Kết quả chạy testcase 3 với lệnh `cat out.txt | sort -r`

🔧 Giải thích kết quả:

- Trong ví dụ này, chương trình tách lệnh **cat out.txt** và **sort -r**, rồi thực hiện hai tiến trình tương tự như các ví dụ trên. Tiến trình p2 thực thi cat, ghi nội dung **file out.txt** vào pipe, còn p1 thực thi sort -r để sắp xếp ngược nội dung đó theo thứ tự giảm dần.
- Kết quả là nội dung trong **out.txt** được in ra màn hình nhưng theo thứ tự đảo ngược. Nhờ cơ chế pipe, dữ liệu được truyền giữa hai tiến trình mà không cần file trung gian.

5. Kết thúc lệnh đang thực thi bằng tổ hợp phím Ctrl + C

- Trong shell it007sh, khi người dùng nhập một lệnh như top, hoặc ping, các lệnh này sẽ thực thi liên tục cho đến khi bị ngắt bằng tổ hợp phím Ctrl + C. Để xử lý tình huống đó, chương trình đã sử dụng hàm **signal()** trong hàm **main()** như sau:

```
signal(SIGINT, control_sig);
```

Hình 24: Câu lệnh chứa hàm xử lý tín hiệu

- Việc này giúp ta tạo một hàm xử lý tín hiệu (control_sig) cho tín hiệu SIGINT (gửi khi nhấn Ctrl + C). Hàm **control_sig()** đơn giản chỉ in một dòng trống và làm sạch bộ đệm xuất (fflush(stdout)), giúp shell không bị thoát mà tiếp tục hiện dấu nhắc it007sh> để người dùng nhập lệnh mới.

- Khi có dòng xử lý **signal(SIGINT, control_sig);**, lúc nhấn Ctrl + C thì **chỉ lệnh con đang thực thi bị ngắt**, còn shell vẫn chạy bình thường. Tuy nhiên, **nếu dòng này bị xóa hoặc comment đi (coi ví dụ bên dưới để thấy rõ)**, khi nhấn Ctrl + C, tín hiệu SIGINT sẽ không được xử lý và chương trình sẽ **thoát hoàn toàn**, bao gồm cả shell – không còn hiển thị dấu nhắc `it007sh>` nữa.

- Testcase1

```

vudaiduong-23520359@VuDuong-32:~/OS_ubuntu/TH/lab6$ ./m
it007sh>top
top - 11:56:52 up 1:14, 2 users, load average: 0.01, 0.04, 0.00
Tasks: 38 total, 1 running, 37 sleeping, 0 stopped, 0 zombie
%Cpu(s): 2.2 us, 0.0 sy, 0.0 ni, 97.8 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 3852.0 total, 2039.3 free, 1704.8 used, 336.8 buff/cache
MiB Swap: 1024.0 total, 1024.0 free, 0.0 used. 2147.2 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM    TIME+  COMMAND
  530 vudaidu+  20   0 1140532 77404 41416 S   10.0   2.0   0:09.79 node
    1 root      20   0  21684  12992  9576 S    0.0   0.3   0:00.93 systemd
    2 root      20   0   2776   1924  1796 S    0.0   0.0   0:00.00 init-systemd(Ub
    7 root      20   0   2776    132   132 S    0.0   0.0   0:00.00 init
   53 root      19  -1  66840  16008 14828 S    0.0   0.4   0:00.70 systemd-journal
   96 root      20   0  23860   5996  4868 S    0.0   0.2   0:00.41 systemd-udevd
  180 systemd+  20   0  21452  11888  9696 S    0.0   0.3   0:00.15 systemd-resolve
  181 systemd+  20   0  91020   6416  5564 S    0.0   0.2   0:00.24 systemd-timesyn
  193 root      20   0   4236   2664  2436 S    0.0   0.1   0:00.01 cron
  194 message+  20   0   9532   5204  4576 S    0.0   0.1   0:00.32 dbus-daemon
  202 root      20   0   18168  8232  7212 S    0.0   0.2   0:00.18 systemd-logind
  205 root      20   0 1756096 16020  9436 S    0.0   0.4   0:00.34 wsl-pro-service
  210 root      20   0    3160   1076   992 S    0.0   0.0   0:00.00 agetty
  213 root      20   0    3116   1188  1096 S    0.0   0.0   0:00.00 agetty
  216 syslog    20   0  222508   5252  4396 S    0.0   0.1   0:00.15 rsyslogd
  217 root      20   0   12020   7488  6416 S    0.0   0.2   0:00.00 sshd
  239 root      20   0  107012  22304 12964 S    0.0   0.6   0:00.12 unattended-upgr
  252 root      20   0   2776    208    80 S    0.0   0.0   0:00.00 SessionLeader
  253 root      20   0   2776    208    80 S    0.0   0.0   0:00.00 Relay(254)
  254 vudaidu+  20   0    6204   5348  3612 S    0.0   0.1   0:00.04 bash
  255 root      20   0    6692   4592  3812 S    0.0   0.1   0:00.00 login

vudaiduong-23520359@VuDuong-32:~/OS_ubuntu/TH/lab6$

```

Hình 25: Kết quả chạy lệnh `top` khi ta comment/xóa dòng code ở hình 24

```

vudaiduong-23520359@VuDuong-32:~/OS_ubuntu/TH/lab6$ ./m
it007sh>top
top - 11:57:50 up 1:15, 2 users, load average: 0.13, 0.07, 0.01
Tasks: 39 total, 1 running, 38 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni,100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 3852.0 total, 1909.6 free, 1811.7 used, 359.8 buff/cache
MiB Swap: 1024.0 total, 1024.0 free, 0.0 used. 2040.4 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM    TIME+  COMMAND
    1 root        20   0   21684   12992   9576 S   0.0   0.3   0:00.94 systemd
    2 root        20   0    2776    1924    1796 S   0.0   0.0   0:00.00 init-systemd(Ub
    7 root        20   0    2776     132     132 S   0.0   0.0   0:00.00 init
   53 root        19  -1   66840   16012   14832 S   0.0   0.4   0:00.71 systemd-journal
   96 root        20   0   23860    5996   4868 S   0.0   0.2   0:00.41 systemd-udev
  180 systemd+    20   0   21452   11888   9696 S   0.0   0.3   0:00.15 systemd-resolve
  181 systemd+    20   0   91020    6416   5564 S   0.0   0.2   0:00.24 systemd-timesyn
  193 root        20   0    4236    2664   2436 S   0.0   0.1   0:00.01 cron
  194 message+    20   0    9532    5204   4576 S   0.0   0.1   0:00.33 dbus-daemon
  202 root        20   0    18168    8232   7212 S   0.0   0.2   0:00.18 systemd-logind
  205 root        20   0 1756096   16020   9436 S   0.0   0.4   0:00.34 wsl-pro-service
  210 root        20   0    3160    1076    992 S   0.0   0.0   0:00.00 agetty
  213 root        20   0    3116    1188   1096 S   0.0   0.0   0:00.00 agetty
  216 syslog      20   0  222508    5252   4396 S   0.0   0.1   0:00.15 rsyslogd
  217 root        20   0   12020    7488   6416 S   0.0   0.2   0:00.00 sshd
  239 root        20   0  107012   22304  12964 S   0.0   0.6   0:00.12 unattended-upgr
  252 root        20   0    2776     208     80 S   0.0   0.0   0:00.00 SessionLeader
  253 root        20   0    2776     208     80 S   0.0   0.0   0:00.00 Relay(254)
  254 vudaidu+    20   0    6204    5348   3612 S   0.0   0.1   0:00.04 bash
  255 root        20   0    6692    4592   3812 S   0.0   0.1   0:00.00 login
  340 vudaidu+    20   0   20260   11408   9324 S   0.0   0.3   0:00.13 systemd
  341 vudaidu+    20   0   21148    1728     0 S   0.0   0.0   0:00.00 (sd-pam)
  352 vudaidu+    20   0    6072    5172   3504 S   0.0   0.1   0:00.01 bash
  395 root        20   0   14736   10216   8532 S   0.0   0.3   0:00.03 sshd
  424 vudaidu+    20   0   15120    6964   4912 S   0.0   0.2   0:07.65 sshd
  425 vudaidu+    20   0     280     198     175 S   0.0   0.0   0:00.03 sh
  443 vudaidu+    20   0   48480   20992  13244 S   0.0   0.5   0:17.82 code-848b80aeb5

it007sh>

```

Hình 26: Kết quả chạy lệnh top khi có hàm xử lý signal

- Testcase 2

```

vudaiduong-23520359@VuDuong-32:~/OS_ubuntu/TH/lab6$ ./m
it007sh>tail -f vduong.txt
Vu Dai Duong 23520359 IT007.P28.2^C
vudaiduong-23520359@VuDuong-32:~/OS_ubuntu/TH/lab6$

```

Hình 27: Kết quả chạy lệnh tail -f vduylab6.txt khi không có hàm xử lý signal

```

it007sh>tail -f vduong.txt
Vu Dai Duong 23520359 IT007.P28.2^C
it007sh>

```

Hình 28: Kết quả chạy lệnh tail -f vduylab6.txt khi có hàm xử lý signal